

Get Shit Done: A meta-prompting, context engineering and spec-driven dev system (github.com/gsd-build)

465 points by stefankuehnel 2 days ago | hide | past | favorite | 252 comments

help

gtirloni 2 days ago | next [-]

I was using this and superpowers but eventually, Plan mode became enough and I prefer to steer Claude Code myself. These frameworks are great for fire-and-forget tasks, especially when there is some research involved but they burn 10x more tokens, in my experience. I was always hitting the Max plan limits for no discernable benefit in the outcomes I was getting. But this will vary a lot depending on how people prefer to work.

[reply](#)

marcus_holmes 2 days ago | parent | next [-]

I ended up grafting the brainstorm, design, and implementation planning skills from Superpowers onto a Ralph-based implementation layer that doesn't ask for my input once the implementation plan is complete. I have to run it in a Docker sandbox because of the dangerously set permissions but that is probably a good idea anyway.

It's working, and I'm enjoying how productive it is, but it feels like a step on a journey rather than the actual destination. I'm looking forward to seeing where this journey ends up.

[reply](#)

LogicFailsMe 2 days ago | root | parent | next [-]

I find simple Ralph loops with an implementer and a reviewer that repeat until everything passes review and unit tests is 90% of the job.

I would love to do something more sophisticated but it's ironic that when I played both agents in this loop over the past few decades, the loop got faster and faster as computers got faster and faster. Now I'm back to waiting on agentic loops just like I used to wait for compilations on large code bases.

[reply](#)

hatmanstack 2 days ago | root | parent | next [-]

Curious what you mean by "played both agents" and "faster and faster"? API calls are API Calls or are you running an open-source model locally?

[reply](#)

gavinray 2 days ago | root | parent | next [-]

Rephrasing of the post in case it's clearer:

"I would love to do something more sophisticated, but it's ironic that when I performed both of the duties done nowadays by agents, the development loop got faster and faster as computers got faster and faster."

[reply](#)

dotancohen 23 hours ago | root | parent | next [-]

For context and curiosity, are you using local inference? Which models?

[reply](#)

auggierose 2 days ago | root | parent | prev | next [-]

If it is working, why is it just a step on a journey? What is missing?

[reply](#)

marcus_holmes 1 day ago | root | parent | next [-]

It's a kludged-together dev process made up of two different systems in a docker container so potential damage is contained. It's not ideal ;)

Neither of those two systems feel evolved either. Superpowers is very cool, but there are holes still. And Ralph feels like an experiment that worked so they published it.

This is all going somewhere, evolving and moving towards some beautiful system. Or maybe the usual dev ecosystem shit - it'll be a great prototype and then it'll get overthought, overcomplicated and overengineered and end up less usable than what we had before *glares at React*

[reply](#)

jghn 2 days ago | root | parent | prev | next [-]

did you hand modify the superpowers skills or are you managing this some other way?

[reply](#)

hatmanstack 2 days ago | root | parent | next [-]

For me, I just created my own prompt pipeline, with a nod towards GANs all of the necessary permissions get surfaced so I don't need to babysit it, and all are relatively simple. No need for Yolo or Dangerously setting Permissions.

[reply](#)

marcus_holmes 1 day ago | root | parent | prev | next [-]

yeah, I coped the skills I wanted into a directory, hacked away at them until they did what I wanted, and then added them to the dockerfile for the sandbox

[reply](#)

jghn 2 days ago | parent | prev | next [-]

I've gone the other way recently, shifting from pure plan mode to superpowers. I was reminded of it due to the announcement of the latest version.

It is perhaps confirmation bias on my part but I've been finding it's doing a better job with similar problems than I was getting with base plan mode. I've been attributing this to its multiple layers of cross checks and self-reviews. Yes, I could do that by hand of course, but I find superpowers is automating what I was already trying to accomplish in this regard.

[reply](#)

gtirloni 2 days ago | root | parent | next [-]

Yes, it does help in that way. Maybe I'm still struggling to let go and let AI take the wheel from beginning to end but I enjoy the exploratory part of the whole process (investigating possible solutions, trying theories, doing little spikes, etc, all with CC's assistance). When it's time to actually code, I just let it do its own thing mostly unsupervised. I do spend quite a lot of time on spec writing.

[reply](#)

jghn 2 days ago | root | parent | next [-]

That's part of what I've liked about it over plan mode. Again not a scientific measurement but I feel it's better at interactive brainstorming and researching the big picture with me. And it's built in multiple checkpoints also give me more space to pivot or course correct.

[reply](#)

healsdata 2 days ago | parent | prev | next [-]

Just tried GSD and Plan Mode on the same exact task (prompt in an MD file). Plan Mode had a plan and then base implementation in twenty minutes. GSD ran for hours to achieve the same thing.

I reviewed the code from both and the GSD code was definitely written with the rest of the project and possibilities in mind, while the Claude Plan was just enough for the MVP.

I can see both having their pros and cons depending on your workflow and size of the task.

[reply](#)

Rapzid 2 days ago | parent | prev | next [-]

I use GitHub Copilot and unfortunately there has been a weird regression in the bundled Plan mode. It suddenly, when they added the new plan memory, started getting both VERY verbose in the plan output and also vague in the details. It's adding a lot of step that are like "design" and "figure out" and railroads you into implementation without asking follow-up questions.

[reply](#)

whalesalad 2 days ago | root | parent | next [-]

I find that even with opus 4.6, copilot feels like it's handicapped. I'm not sure if it's related to memory or what but if I give two tasks to opus4.6 one in CC and one in Copilot, CC is substantially better.

I've been really enjoying Codex CLI recently though. It seems to do just as well as Opus 4.6, but using the standard GPT 5.4

[reply](#)

chaostheory 2 days ago | root | parent | next [-]

I have the same experience with Antigravity and Gemini CLI, both using Gemini 3 Pro. CLI works on the problem with more effort and time. Meanwhile, antigravity writes shitty python scripts for a few seconds and calls it a day. The agent harness matters a lot

[reply](#)

Atotalnoob 2 days ago | root | parent | prev | next [-]

Copilot feels like being a caveman, Claude code feels like modern times comparatively.

[reply](#)

gtirloni 2 days ago | root | parent | prev | next [-]

I think this shows that the model alone isn't the complete story and that these "harnesses" (as people seem to be calling them) shape a lot of the experienced behavior of these tools.

[reply](#)

theshrike79 21 hours ago | root | parent | next [-]

My analogy is that the model is the engine and the harness is the driver and chassis.

You can have the biggest monster of an engine ever, but if you put it in a tricycle and a grandma is driving, you won't get good results.

[reply](#)

codebolt 1 day ago | root | parent | prev | next [-]

Opus 4.6 has a 200k context limit in Copilot. Could be the issue.

[reply](#)

nfg 2 days ago | root | parent | prev | next [-]

As a matter of interest are you using the copilot cli?

[reply](#)

whalesalad 2 days ago | root | parent | next [-]

yeah. copilot cli using opus 4.6 vs claude code using opus 4.6

[reply](#)

nfg 1 day ago | root | parent | next [-]

If you could share I'd be really interested in hearing a concrete example of the two behaving differently. I work in Microsoft (not on copilot - though I'm an heavy user, and use Claude code in a personal capacity) and would be quite happy to repro and report back to the copilot cli team who are responsive.

[reply](#)

NSPG911 2 days ago | root | parent | prev | next [-]

> VERY verbose in the plan output

Is that an issue? GitHub charges per-request, not per-token, so a verbose output and short output will be the same cost

What model are you using?

[reply](#)

junker 2 days ago | root | parent | next [-]

The problem might be that our brains charge per token, which makes reviewing hard. :)

[reply](#)

whalesalad 2 days ago | parent | prev | next [-]

Same experience. Superpowers are a little too overzealous at times. For coding especially I don't like seeing a comprehensive design spec written (good) and then turning that into effectively the same doc but macro expanded to become a complete implementation with the literal code for the entire thing in a second doc (bad). Even for trivial changes I'd end up with a good and succinct -design.md, then an -implementation.md, then end with a swarm of sub agents getting into races while more or less just grabbing a block from the implementation file and writing it.

A mess. I still enjoy superpowers brainstorming but will pull the chute towards the end and then deliver myself.

[reply](#)

[gtirloni](#) 2 days ago | root | parent | next [-]

Yes. I sometimes had to specifically ask it to NOT add any code to the specs because that would be done at a later stage.

[reply](#)

[sigbottle](#) 2 days ago | parent | prev | next [-]

Yup yup yup. I burned literally a weeks worth of the 20\$ claude subscription and then 20\$ worth of API credits on gsdv2. To get like 500 LOC.

And that was AFTER literally burning a weeks worth of codex and Claude 20\$ plans and 50\$ API credits and getting completely bumfucked - AI was faking out tests etc.

I had better experiences just guiding the thing myself. It definitely was not a set and forget experience (6 hours of constant monitoring) but I was able to get a full research MVP that informed the next iteration with only 75% of a codex weekly plan.

[reply](#)

[FromTheFirstIn](#) 2 days ago | root | parent | next [-]

You spent \$25 on 500 LOC?

[reply](#)

[sigbottle](#) 2 days ago | root | parent | next [-]

Well, there were milestones and docs and extra scaffolding that the gsd system produces, but yes. and it didn't seem like progress was going to go any faster.

[reply](#)

[SayThatSh](#) 2 days ago | parent | prev | next [-]

I've played around a bit with the plugins and as you've said, plan mode really handles things fine for the most part. I've got various workflows I run through in Claude and I've found having CC create custom skills/agents created for them gets me 80% of the way there. It's also nice that letting the Claude file refer to them rather than trying to define entire workflows within it goes a long way. It'll still forget things here and there, leading to wasted tokens as it realizes it's being dumb and corrects itself, but nothing too crazy. At least, it's more than enough to let me continue using it naturally rather than memorizing a million slash commands to manually evoke.

[reply](#)

[abhisek](#) 2 days ago | parent | prev | next [-]

I have been using superpowers for Gryph development for a while. Love the brainstorming and exploration that it brings in. Haven't really compared token usage but something in my bucket.

[reply](#)

[locknitpicker](#) 2 days ago | parent | prev | next [-]

> I was using this and superpowers but eventually, Plan mode became enough and I prefer to steer Claude Code myself.

Plan mode is great, but to me that's just prompting your LLM agent of choice to generate an ad-hoc, imprecise, and incomplete spec.

The downside of specs is that they can consume a lot of context window with things that are not needed for the task. When that is a concern, passing the spec to plan mode tends to mitigate the issue.

[reply](#)

[hatmanstack](#) 2 days ago | parent | prev | next [-]

Why are we using cli wrappers if you're using Claude Code? I get if you need something like Codex but they released sub agents today so maybe not even that, but it's an unnecessary wrapper for Claude Code.

[reply](#)

[odie5533](#) 2 days ago | root | parent | next [-]

Wrappers are useful for some tasks. I use ralph loops for things that are extremely complicated and take days of work. Like reverse engineering projects or large scale migration efforts.

[reply](#)

[hatmanstack](#) 2 days ago | root | parent | next [-]

Even with the 1 mil context windows? Can't you just keep the orchestrator going and run sub agents? Maybe the added space is too new? I also haven't tested out the context rot from 300K and up. Would love some color on it from first hand exp.

[reply](#)

[odie5533](#) 2 days ago | root | parent | next [-]

It's not a context issue so much as a focus issue. The agent will complete part of a task and then ask if I want it to continue. Even if I told it I want it to keep going until all tasks are complete. Using a wrapper deals with that behavior.

Most projects I do take 20 minutes or less for an agent to complete and those don't need a wrapper. But for longer tasks, like hours or days, it gets distracted.

[reply](#)

mrhaugan 2 days ago | root | parent | next [-]

Damn, what kind of tasks are you making your agents work on that takes days???

[reply](#)

odie5533 1 day ago | root | parent | next [-]

Claude Code has been working 24/7 for the past 4 days on creating a private server for a dead video game. It managed to get login, chat, inventory, and a few other features working. I provided it tools like Ghidra and x64dbg and pywinauto. Progress is slow but incremental. Each day new bits work that didn't before.

[reply](#)

roncesvalles 2 days ago | root | parent | prev | next [-]

So that you can have a fresh context for every little thing. These harnesses basically marry LLMs with deterministic software logic. The harness programmatically generates the prompts and stores the output, step by step.

You never want the LLM to do anything that deterministic software does better, because it inflates the context and is not guaranteed to be done accurately. This includes things like tracking progress, figuring out dependency ordering, etc.

[reply](#)

gtirloni 2 days ago | root | parent | prev | next [-]

GSD and superpowers aren't CLI wrappers?

[reply](#)

hatmanstack 2 days ago | root | parent | next [-]

It's a cli wrapper. Don't know how you could say it wasn't.

edit: GSD is a cli wrapper, Superpowers not so much. Both are over-engineered for an easy problem IMHO.

[reply](#)

ramoz 2 days ago | root | parent | next [-]

Both are dramatically over-engineered. & That's okay. I find them to be products of an industry reconciling how to really work with AI as well as optimize workflows around it. Similar to Gastown et al.

Otherwise, if you can own your own thinking, orchestrating, and steering of agents, you're in a more mature place.

[reply](#)

mycall 2 days ago | root | parent | next [-]

I also see it as fleeting as right when you have it figured out, a new model will work differently and may/may not need all their engineering layers.

[reply](#)

hatmanstack 2 days ago | root | parent | prev | next [-]

I think that's fair, if they were created today I'm sure the creators would make different decisions, a penalty of getting there first.

[reply](#)

hermanzegerman 2 days ago | root | parent | prev | next [-]

No it's not. It's using Skills and Agents and runs always inside of Claude Code, Gemini CLI etc...

[reply](#)

swingboy 2 days ago | root | parent | next [-]

GSD delegates a lot of the deterministic work to a JavaScript CLI. That might be what the poster is talking about.

[reply](#)

gtirloni 2 days ago | root | parent | next [-]

That's definitely not a CLI wrapper. But people are calling Claude Code (clearly a TUI) a CLI so :shrug:

GSD is a collection of skills, commands, MCPs(?), helper scripts, etc that you use *inside* Claude Code (and others). If anything, Claude Code is the wrapper around those things and not the other way around.

Re: helper scripts. Anyone doing extensive work in any AI-assisted platform has experienced the situation where the agent wants to update 10k files individually and it takes ages. CC is often smart enough to code a quick Python script for those changes and the GSD helper scripts help in the same way. It's just trying to save tokens. Hardly a wrapper around Claude Code.

[reply](#)

andai 2 days ago | parent | prev | next [-]

What's happening with the other 90%?

[reply](#)

joegaebel 2 days ago | prev | next [-]

In my view, Spec-Driven systems are doomed to fail. There's nothing that couples the english language specs you've written with the actual code and behaviour of the system - unless your agent is being insanely diligent and constantly checking if the entire system aligns with your specs.

This has been solved already - automated testing. They encode behaviour of the system into executables which actually tell you if your system aligns or not.

Better to encode the behaviour of your system into real, executable, scalable specs (aka automated tests), otherwise your app's behaviour is going to spiral out of control after the Nth AI generated feature.

The way to ensure this actually scales with the firepower that LLMs have for writing implementation is ensure it follows a workflow where it knows how to test, it writes the tests first, and ensures that the tests actually reflect the behaviour of the system with mutation testing.

I've scoped this out here [1] and here [2].

[1] <https://www.joegaebel.com/articles/principled-agentic-softwa...> [2] <https://github.com/JoeGaebel/outside-in-tdd-starter>
[reply](#)

oakpond 2 days ago | parent | next [-]

Sort of agreed. Natural language specs don't scale. They can't be used to accurately model and verify the behavior of complex systems. But they can be used as a guide to create formal language specs that can be used for that purpose. As long as the formal spec is considered to be the ground truth, I think it can scale. But yeah, that means some kind of code will be required.. :)

[reply](#)

j45 1 day ago | root | parent | next [-]

Things like Github's speckit seems to have a fair amount of usage.

The idea that specs are code now, is one can effectively rebuild in the future with newer models. Test requirements could be defined upfront in the specs too, no?

[reply](#)

oakpond 1 day ago | root | parent | next [-]

I think natural language leaves too much room for ambiguities. If you treat it as code I expect you will run into frequent bugs and unintended side effects of LLM-authored changes as your software evolves. So I'm skeptical about this approach.

A formal language helps in this regard because it makes visible the inconsistencies that are hidden in the specifications.

Coding is difficult sometimes because it turns out the problem you are trying to solve is more difficult than expected (not because it's difficult to code).

[reply](#)

j45 1 day ago | root | parent | next [-]

Sounds like this perspective is theoretical.

Been building for a long time, and more specifically overseeing building in detail, which transfers interestingly to overseeing LLMs.

Just like with coworkers, providing the right amount of context (not too much, or too little) for the request to succeed is critical.

I shared similar views, but I have seen first hand (using in production myself) that specs, well done in a way for LLMs, can do development with AI that works. If something doesn't work out, you don't fix the code, you adjust the spec. Highly recommend watching doers on Youtube who are sharing screens.

Discovering a problem is more difficult than expected allows you to take more shots at it, quicker by adjusting the spec, for example and running again. We are used to just plowing ahead to make the code right, instead of improving/clarifying the ask/spec.

[reply](#)

oakpond 1 day ago | root | parent | next [-]

In my experience, when you sell expensive complex systems, customers are very worried about any differences in system behavior as a result of software updates.

When you implement a new feature with these tools, how do you convince yourself that existing system behavior remains unchanged?

When you have the code in front of you, atleast you can reason about the full system behavior before and after because code is unambiguous like that.

With spec driven development, the LLM can rewrite anything as long as it meets the spec. That's a problem if your customer relies on behavior that's written down ambiguously (or omitted entirely).

So, I think this is only going to work if you write specs with mathematical precision.. at which point you probably want to write them using a mathematical language.

[reply](#)

j45 23 hours ago | root | parent | next [-]

Appreciate learning from your perspective.

I've built, integrated and sold expensive complex systems. They want it working, connected, and reliable. Lots of paths there.

Have you built with LLMs? I'm asking because I would refer to things from having something working on a complex code base.

Specifications, or inputs in a way are a new code. The added focus on documentation, before and after is a bonus too, and also helps with alignment.

Code styles/formats/philosophies can be documented and followed.

The human process of what to look into, in what way, for what areas of the code base, can also be trained and remembered. There are ways to achieve and maintain precision without 100% mathematical precision, because there are only so many ways to solve a problem, or step and the mechanisms for deciding can also be defined in general, or specific.

[reply](#)

oakpond 10 hours ago | root | parent | next [-]

I build with LLMs all the time but I generally don't do vibe coding unless it's something small I don't really care about.

When I look at Speckit, I see a kind of vibe coding fantasy: "code is no longer king", stop writing "undifferentiated code." There is no code on the site, just a bunch of prompts and commands.

On the other hand, what you are describing above is bringing specs closer to the codebase, while not replacing the code itself. Like I said I have no problems using natural language as a guide (even as a primary guide). I also completely agree that it helps with documentation.

My main point is: if you want to maintain a complex system, you also need to have an accurate description of the system behavior in some kind of formalism.

This kind of description reflects the true system behavior better. It's more helpful when you need to predict the impact of changes and also during debugging.

[reply](#)

internet_points 2 days ago | parent | prev | next [-]

See also recent post "A sufficiently detailed spec is code" which tried and failed to reproduce openai's spec results: <https://hn.algolia.com/?q=https%3A%2F%2Fhaskellforall.com%2F...>

[reply](#)

zby 2 days ago | parent | prev | next [-]

Spec Driven Development is a curious term - it suggests it is a kind of, or at least in the tradition of, Test Driven Development but it goes in the opposite direction!

[reply](#)

sveme 2 days ago | root | parent | next [-]

Don't understand this - you can go spec -> test -> implementation and establish the test loop. Bit like the v model of old, actually.

[reply](#)

joegaebel 2 days ago | root | parent | next [-]

In my view, the problem with specs are:

1. Specs are subject to bit-rot, there's no impetus to update them as behaviour changes - unless your agent workflow explicitly enforces a thorough review and update of the specs, and unless your agent is diligent with following it. Lots of trust required on your LLM here.
2. There's no way to systematically determine if the behaviour of your system matches the specs. Imagine a reasonable sized codebase - if there's a spec document for every feature, you're looking at quite a collection of specs. How many tokens need be burnt to ensure that these specs are always up to date as new features come in and behaviour changes?
3. Specs are written in English. They're ambiguous - they can absolutely serve the planning and design phases, but this ambiguity prevents meaningful behaviour assertions about the system as it grows.

Contrast that with tests:

1. They are executable and have the precision of code. They don't just describe behaviour of the system, they validate that the system follows that behaviour, without ambiguity.
2. They scale - it's completely reasonable to have extensive codebases have all (if not most) of their behaviour covered by tests.
3. Updating is enforceable - assuming you're using a CI pipeline, when tests break, they must be updated in order to continue.
4. You can systematically determine if the tests fully describe the behaviour (ie. is all the behaviour tested) via mutation testing. This will tell you with absolute certainty if code is tested or not - do the tests fully describe the system's behaviour.

That being said, I think it's very valuable to start with a planning stage, even to provide a spec, such that the correct behaviour gets encoded into tests, and then instantiated by the implementation. But in my view, specs are best used within the design stage, and if left in the codebase, treated only as historical info for what went into the development of the feature. Attempting to use them as the source of truth for the behaviour of the system is fraught.

And I guess finally, I think that insofar as any framework uses the specs as the source of truth for behaviour, they're going to run into alignment problems since maintaining specs doesn't scale.

[reply](#)

zby 2 days ago | root | parent | prev | next [-]

SDD is about flowing the design choices from the spec into the rest of the system. TDD was for making sure that the inevitable changes you make to the system later don't break your earlier assumptions - or at least warn that you need to change them. Personally I don't buy TDD - it might be useful sometimes - but it is kind of extreme - but in general agile methodologies were a reaction to the waterfall model of system development.

[reply](#)

anthonyrstevens 2 days ago | root | parent | next [-]

This is just one way to use TDD. I personally get the most value from TDD as a design approach. I iteratively decompose the project into stubbed, testable components as I start the project, and implement when I have to to get my tests to pass. At each stage I'm asking myself questions like "who needs to call who? with what data? What does it expect back as a return value?" etc.

[reply](#)

j45 2 days ago | parent | prev | next [-]

Specs see more about alignment and clarity increasing code that works, and increase the success of tests.

[reply](#)

locknitpicker 2 days ago | parent | prev | next [-]

> This has been solved already - automated testing.

This is specious reasoning. Automated tests are already the output of these specs, and specs cover way more than what you cover with code.

Framing tests as the feedback that drives design is also a baffling opinion. Without specialized prompts such as specs, you LLM agent of choice ends up either ignoring tests altogether or even changing them to fit their own baseless assumptions.

I mean, who hasn't stumbled upon the infamous "the rest of your tests go here" output in automated tests?

[reply](#)

polytely 2 days ago | root | parent | next [-]

> Automated tests are already the output of these specs, and specs cover way more than what you cover with code.

ok but how are you sure that the AI is correctly turning the spec into tests. if it makes a mistake there and then builds the code in accordance with the mistaken test you only get the Illusion of a correct implementation

[reply](#)

locknitpicker 2 days ago | root | parent | next [-]

> ok but how are you sure that the AI is correctly turning the spec into tests.

You use the specs to generate the tests, and you review the changes.

[reply](#)

mattmanser 2 days ago | root | parent | prev | next [-]

I've seen a few comments recently that start with:

This is specious reasoning

It's an insulting phrase and from now on I'm immediately down voting it when I see it.

[reply](#)

nelox 2 days ago | root | parent | next [-]

On the face of it is insulting, until you dig a little deeper

[reply](#)

locknitpicker 2 days ago | root | parent | prev | next [-]

> It's an insulting phrase (...)

I'm sorry you feel like that. How would you phrase an observation where you find the rationale for an assertion to not be substantiated and supported beyond surface level?

[reply](#)

coopykins 2 days ago | prev | next [-]

There are so many of these "meta" frameworks going around. I have yet to see one that proves in any meaningful way they improve anything. I have a hard time believing they accomplish anything other than burn tokens and poison the context window with too much information. What works best IME is keeping things simple, clear and only providing the essential information for the task at hand, and iterating in manageable slices, rather than trying to one-shot complex tasks. Just Plan, Code and Verify, simple as that.

[reply](#)

wffurr 2 days ago | parent | next [-]

There was a post from Apenwarr[1] recently that gave it a name: "the AI Developer's Descent Into Madness", ending with "I need an agent framework. I can have my agent write an agent framework!"

[1]: <https://apenwarr.ca/log/20260316>

[reply](#)

hnthrow0287345 2 days ago | root | parent | next [-]

Sounds like a FactoryFactory to me which tells me Java developers have always been mad

[reply](#)

dboreham 2 days ago | parent | prev | next [-]

From my experience they are motivated by these two issues that you run into when using Claude Code (or similar tool):

1. The LLM is operating on more what you'd call "guidelines" than the rules -- it will mostly make a PR after fixing a bug, but sometimes not. It will mostly run tests after completing a fix, but sometimes not. So there's a sentiment "heck, let's write some prompt that tells it to always run tests after fixing code", etc.
2. You end up running the LLM tool against state that is in GitHub (or RCS du jour). E.g. I open a bug (issue) and type what I found that's wrong, or whatever new feature I want. Then I tell Claude to go look at issue #xx. It runs in the

terminal, asks me a bunch of unnecessary permission questions, fixes the bug, then perhaps makes a PR, perhaps I have to ask for that, then I go watch CI status on the PR, come back to the terminal and tell it that CI passed so please merge (or I can ask it to watch CI and review status and merge when ready). After a while you realize that all that process could just be driven from the GitHub UI -- if there was a "have Claude work on this issue" button. No need for the terminal.

[reply](#)

joshribakoff 2 days ago | root | parent | next [-]

After a while many people then realize this often produces worse results by injecting additional noise in context like the overhead of invoking the gh cli and parsing json comments or worse the mcp.

But they get the dopamine loop of keeping the loop alive, flashing colors, high score/token use, and plausible looking outputs — so its easy to deceive oneself into thinking something remarkable was discovered

[reply](#)

quangtrn 2 days ago | parent | prev | next [-]

The structured spec approach has worked well for me — but only when the spec itself is visual, not more text. I've been designing app navigation flows as screen images with hotspot connections, then exporting that as structured markdown. The AI gets screen-by-screen context instead of one massive prompt. The difference vs writing the spec by hand is that the visual layout catches gaps (orphan screens, missing error states) before you hand anything to the LLM.

[reply](#)

mingyeow 2 days ago | root | parent | next [-]

How does that work- mind sharing your workflow?

[reply](#)

quangtrn 2 days ago | root | parent | next [-]

It's a tool called Drawd (drawd.app). You upload screen mockups onto an infinite canvas, draw rectangles over tap areas to define hotspots, then connect screens with arrows to map the navigation flow. Each hotspot carries an action type — navigate, API call, modal, conditional branch. When you're done, it exports structured markdown files (screen inventory, navigation map, build guide) that you feed to the LLM as context. The visual step is what catches gaps before you burn tokens on a broken spec.

[reply](#)

romanovcode 2 days ago | parent | prev | next [-]

It's basically .vimrc/.emacs.d of the current age.

These meta-frameworks are useful for the one who set them up but for another person they seem like complete garbage.

[reply](#)

petesergeant 2 days ago | parent | prev | next [-]

I have my own mini framework that marries Claude and Codex. When I see the clangers that Claude by itself produces that Codex catches, I can't see how I'd ever just let a single agent do its thing.

[reply](#)

neebz 2 days ago | parent | prev | next [-]

Once the plan stage is done is it fire-and-forget for you afterwards?

[reply](#)

AndyNemmy 2 days ago | prev | next [-]

I have a ai system i use. I'd like to release it so others can benefit, but at the same time it's all custom to myself and what i do, and work on.

If I fork out a version for others that is public, then I have to maintain that variation as well.

Is anyone in a similar situation? I think most of the ones I see released are not particularly complex compreaed to my system, but at the same time I don't know how to convey how to use my system as someone who just uses it alone.

it feels like I don't want anyone to run my system, I just want people to point their ai system to mine and ask it what there is valuable to potentially add to their own system.

I don't want to maintain one for people. I don't want to market it as some magic cure. Just show patterns that others can use.

[reply](#)

canadiantim 2 days ago | parent | next [-]

you don't have to maintain it. Especially in the age of ai, just giving people inspiration and something to vibe from is more than sufficient and appreciated

[reply](#)

AndyNemmity 2 days ago | root | parent | next [-]

alright. i guess i'll create a new repo, remove out a bunch of very specific pieces, and put it up.

there's a lot of patterns i think are helpful for me.

[reply](#)

canadiantim 2 days ago | root | parent | next [-]

That would be awesome. I believe with AI it's all about tailoring everything to your specific workflow and style, especially anything to do with the dev environment.

[reply](#)

AndyNemmity 2 days ago | root | parent | next [-]

right, i'm having to cut out a lot of my pieces at the moment to try to get it into a release state.

i have checks of which types of repos i'm in with branching dev flows for each one.

it's going to be hard to communicate all of this generically, but i am trying.

[reply](#)

canadiantim 1 day ago | root | parent | next [-]

No worries if it's untenable or too much though, but I'll keep an eye on the commend thread in case!

For my part, I'm currently using oh-my-opensource harness with various skills extracted and tailored from superpowers / simonw / matt pocock. Working well enough so far, but keen to really evolve the skill flow and how they connect and are used in coordination with the various subagents.

[reply](#)

AndyNemmity 1 day ago | root | parent | next [-]

I ended up releasing it. Let me know your thoughts

<https://github.com/notque/claude-code-toolkit>

[reply](#)

tensegrist 2 days ago | root | parent | prev | next [-]

if it's on github you could even archive it from the get-go

[reply](#)

maccam912 2 days ago | prev | next [-]

I've had a good experience with <https://github.com/obra/superpowers>. At first glance this looks similar. Has anyone tried both who can offer a comparison?

[reply](#)

yolonir 2 days ago | parent | next [-]

I've used both From my experience, gsd is a highly overengineered piece of software that unfortunately does not get shit done, burns limits and takes ages while doing so. Quick mode does not really help because it kills the point of gsd, you can't build full software on ad-hocs. I've used plain markdown planning before, but it was limiting and not very stable, superpowers looks like a good middleground

[reply](#)

esperent 2 days ago | root | parent | next [-]

> gsd is a highly overengineered piece of software that unfortunately does not get shit done, burns limits and takes ages while doing so

That was my impression of superpowers as well. Maybe not *highly* overengineered but definitely *somewhat*. I ended up stripping it back to get something useful. Kept maybe 30%.

There's a kernel of a good idea in there but I feel it's something that we're all gradually aligning on independently, these shared systems are just fancy versions of a "standard agentic workflow".

[reply](#)

dmix 2 days ago | root | parent | prev | next [-]

My instinct is to blame these agent frameworks as well but at some point we have to start blaming Claude or Claude Code for engaging in these endless planning loops which burn tokens with no regard. The future of these coding models will eventually need to start factoring in how to use and engage with these skills more competently (assuming that's possible and they aren't always just aimless yesmen).

[reply](#)

hrrmtst93837 2 days ago | root | parent | prev | next [-]

Superpowers looks more like PM-in-a-box with AI paint. If you want speed, thin scripts plus sane CLI tools and Git hooks will get you further in an afternoon than these 'meta' systems, because they still depend on manual curation for anything nontrivial and mostly burn limits while shuffling context around.

[reply](#)

observationist 2 days ago | parent | prev | next [-]

It's one of those things where having a structure is really helpful - I've used some similar prompt scaffolds, and the difference is very noticeable.

Another great technique is to use one of these structures in a repo, then task your AI with overhauling the framework using best practices for whatever your target project is. It works great for creative writing, humanizing, songwriting, technical/scientific domains, and so on. In conjunction with agents, these are excellent to have.

I think they're going to be a temporary thing - a hack that boosts utility for a few model releases until there's sufficient successful use cases in the training data that models can just do this sort of thing really well without all the extra prompting.

These are fun to use.

[reply](#)

huydotnet 2 days ago | parent | prev | next [-]

I've tried both. Each has pros and cons. Two things I don't like about superpowers is it writes all the codes into the implementation plan, at the plan step, then the subagents basically just rewrite these codes back to the files. And I have to ask Claude to create a progress.md file to track the progress if I want to work in multiple sessions. GSD pretty much solved these problems for me, but the down side of GSD is it takes too many turns to get something done.

[reply](#)

denolfe 2 days ago | root | parent | next [-]

There is a fork that uses Claude Code-native features and tracks progress and task dependencies natively: <https://github.com/pcvelz/superpowers>

[reply](#)

jghn 2 days ago | root | parent | next [-]

If you use it I'm curious if you find it limited at all from lagging behind superpowers? For instance I opened up one skill at random and they haven't yet pulled in the latest commit from last week.

I doubt any hot off the press features are *that* important, but am curious if the customizations of the fork are a net positive considering this.

[reply](#)

annjose 2 days ago | parent | prev | next [-]

I tried Superpowers for my current project - migrating my blog from Hugo to Astro (with AstroPaper theme). I wrote the main spec in two ways - 1) my usual method of starting with a small list of what I want in the new blog and working with the agent to expand on it, ask questions and so on (aka Collaborative Spec) and 2) asked Superpowers to write the spec and plan. I did both from the working directory of my blog's repo so that the agent has full access to the code and the content.

My findings:

1. The spec created by Superpowers was very detailed (described the specific fonts, color palette), included the exact content of config files, commit messages etc. But it missed a lot of things like analytics, RSS feed etc.
2. Superpowers wrote the spec and plan as two separate documents which was better than the collaborative method, which put both into one document.
3. Superpowers recommended an in-place migration of the blog whereas the collaborative spec suggested a parallel branch so that Hugo and Astro can co-exist until everything is stable.

And a few more difference written in [0].

In general, I liked the aspect of developing the spec through discussion rather than one-shotting it, it let me add things to the spec as I remember them. It felt like a more iterative discovery process vs. you need to get everything right the first time. That might just be a personal preference though.

At the end of this exercise, I asked Claude to review both specs in detail, it found a few things that both specs missed (SEO, rollback plan etc.) and made a final spec that consolidates everything.

[0] <https://annjose.com/redesign/#two-specs-one-project>

[reply](#)

patates 2 days ago | root | parent | next [-]

I usually ask Gemini to review the spec as well. Sometimes it catches things I missed even after going through a few times.

[reply](#)

jimmySixDOF 2 days ago | root | parent | prev | next [-]

I'm a big fan of Research Plan Implement like this peak build-in-public multi foundation model cross check approach:

<https://x.com/i/status/2033368385724014827>

[reply](#)

hatmanstack 2 days ago | parent | prev | next [-]

I don't get why people need a cli wrapper for this. Can't you just use Claude skills and create everything you need?

[reply](#)

1_1xdev1 2 days ago | root | parent | next [-]

What do you mean by cli wrapper?

Superpowers and gsd are claude code plugins (providing skills)

[reply](#)

fcatalan 2 days ago | root | parent | prev | next [-]

Superpowers is literally a bunch of skills packaged in a Claude plugin

[reply](#)

hatmanstack 2 days ago | root | parent | next [-]

Right on, I was going off the OP's GSD link, which looks like the def of a cli wrapper to me. Hadn't seen superpowers before, seems way too deterministic and convoluted, but you're right, not a cli wrapper.

[reply](#)

darthwalsh 1 day ago | root | parent | next [-]

There's a CLI tool that writes the agent skills into the right folder. The other option would be to have everybody manually unzip a download into a folder which they might not remember.

[reply](#)

CharlesW 2 days ago | parent | prev | next [-]

Yes, and IMO Superpowers is better when you want to Get Not-Shit Done.

Get Shit Done is best when when you're an influencer and need to create a Potemkin SaaS overnight for tomorrow's TikTok posts.

[reply](#)

yoaviram 2 days ago | prev | next [-]

I've been using GSD extensively over the past 3 months. I previously used speckit, which I found lacking. GSD consistently gets me 95% of the way there on complex tasks. That's amazing. The last 5% is mostly "manual" testing. We've used GSD to build and launch a SaaS product including an agent-first CMS (whiteboar.it).

It's hard to say why GSD worked so much better for us than other similar frameworks, because the underlying models also improved considerably during the same period. What is clear is that it's a huge productivity boost over vanilla Claude Code.

[reply](#)

unstatusthequo 2 days ago | parent | next [-]

Same. Have had great results with it. I got sick of paying FreshBooks monthly for basic income/expense tracking for Schedule C reporting and used GSD to build a macOS Swift app with Codex 5.4 and Opus 4.6. It's working great and I am considering releasing it on the App Store. It started as a web app, but then I wanted screen capture from other windows for receipts in email or whatever. Then I wanted physical receipts, and so used Apple continuity camera. All working now in my app. And, I just added receipt auto-extract to pull salient info from and determine deduction category using Anthropic API.

Yes this is how much paying FreshBooks annoyed me. Plus I hated they forced an emailed 2FA if you didn't connect with Google.

[reply](#)

wg0 2 days ago | root | parent | next [-]

How much feature complete it is compared to Freshbooks?

Also, how much it is in terms of cost? Like - API costs?

Is it pure Swift? Or Electron app?

[reply](#)

Frannky 2 days ago | prev | next [-]

I tried it once; it was incredibly verbose, generating an insane amount of files. I stopped using it because I was worried it would not be possible to rapidly, cheaply, and robustly update things as interaction with users generated new requirements.

The best way I have today is to start with a project requirements document and then ask for a step-by-step implementation plan, and then go do the thing at each step but only after I greenlight the strategy of the current step. I also specify minimal, modular, and functional stateless code.

[reply](#)

toastal 2 days ago | prev | next [-]

This pile of Markdown files has the most cringe-inducing name I have seen in weeks.

[reply](#)

sunnyps 2 days ago | parent | next [-]

Surely gstack was worse, right?

[reply](#)

barbazoo 2 days ago | parent | prev | next [-]

Having md files under "Languages" would be nice here.

[reply](#)

cush 1 day ago | parent | prev | next [-]

fwiw GSD is a pretty well-established decision-making framework. The creator didn't invent it

[reply](#)

btiwaree 2 days ago | prev | next [-]

I used this for a team hackathon and it took way too much time to build understanding of the codebase, wrote too many agent transcripts and spent way too much token during generation. It also failed multiple times when either generating agent transcript or extracting things from agent transcript - once citing "The agent transcripts are too complex to extract from" - quite confounding considering it's the transcript you created. For what we were trying to build - few small sets of features - using gsd was an overkill. The idea was to get some learnings whether gsd could be useful - for our case it was a strong no. Learning for me: don't overcomplicate - write better specs, use claude plan mode, iterate.

[reply](#)

DamienB 2 days ago | prev | next [-]

I've compared this to superpowers and the classic prd->task generator. And I came away convinced that less is more. At least at the moment. gsd performed well, but took hours instead of minutes. Having a simple explanation of how to create a PRD followed by a slightly more technical task list performed much better. It wasn't that grd or superpowers couldn't find a solution, it's just that they did it much slower and with a lot more help. For me, the lesson was that the workflow has changed, and we that we can't apply old project-dev paradigms to this new/alien technology. There's a new instruction manual and it doesn't build on the old one.

[reply](#)

gbrindisi 2 days ago | prev | next [-]

I like openspec, it lets you tune the workflow to your liking and doesn't get in the way.

I started with all the standard spec flow and as I got more confident and opinionated I simplified it to my liking.

I think the point of any spec driven framework is that you want to eventually own the workflow yourself, so that you can constraint code generation on your own terms.

[reply](#)

alasano 2 days ago | parent | next [-]

I also like openspec.

I think these type of systems (gsd/superpowers) are way too opinionated.

It's not that they can't or don't work. I just think that the best way to truly stay on top of the crazy pace of changes is to not attach yourself to super opinionated workflows like these.

I'm building an orchestrator library on top of openspec for that reason.

[reply](#)

gbrindisi 2 days ago | root | parent | next [-]

I am doing something similar: I use openspec to create context and a sequential task list that I feed to ralph loops, so that i'm involved for the planning and the verification step but completely hands off the wheel during code generation.

[reply](#)

alasano 1 day ago | root | parent | next [-]

Exactly that. I created an "Open Ralph" loop initially within Claude directly with review gates per phase in the OpenSpec task list.

But it was always just a workaround to what I truly wanted (what I'm building now), a full external managed orchestrator loop. The agents aren't aware of the loop, they don't need to be.

[reply](#)

vinnyamac 2 days ago | prev | next [-]

I tried this for a week and gave up. Required far too much back and forth. Ate too many tokens, and required too much human in the loop.

For this reason I don't think it's actually a good name. It should be called planning-shit instead. Since that's seemingly 80%+ of what I did while interacting with this tool. And when it came to getting things done, I didn't need this at all, and the plans were just alright.

[reply](#)

recroad 2 days ago | prev | next [-]

I use openspec and love it. I'm doing 5-7x with close to 100% of code AI generated, and shipping to production multiple times a day. I work on a large sass app with hundreds of customers. Wrote something here:

<https://zarar.dev/spec-driven-development-from-vibe-coding-t...>

[reply](#)

reedlaw 2 days ago | parent | next [-]

This is the second endorsement I've seen today. I gave OpenSpec a shot and was dismayed by the Explore prompt. [1] Over 1,000 words with verbose, repetitive instructions which will lead to context drift. The examples refer to specific tools like SQLite and OAuth. That won't help if your project isn't related to those.

I do like the basic concept and directory structure, but those are easy enough to adopt without all the craft.

1. <https://github.com/Fission-AI/OpenSpec/blob/main/src/core/te...>

[reply](#)

brendanmc6 2 days ago | parent | prev | next [-]

This is a great post, thanks for sharing! Over the last couple months I fell into my own unique (but similar) spec driven workflow and couldn't help but start building my own tooling around it. Since you've clearly thought so much about this I would really value any feedback / criticism / reactions you have.

<https://acai.sh>

I find the added structure of yaml + requirement ids helps tremendously compared to plain markdown -

<https://acai.sh/writing-specs>

I am still a few days away from open sourcing the stack (CLI / API & Server), plan is to gather as much feedback as I can and decide if this is worth maintaining.

[reply](#)

sarchertech 2 days ago | parent | prev | next [-]

>large sass app

>hundreds of customers

[reply](#)

recroad 2 days ago | root | parent | next [-]

Large codebase. Yeah man, I have a small business, trying to grow but not easy going up against Ticketmaster.

[reply](#)

sarchertech 2 days ago | root | parent | next [-]

A ticketmaster competitor doesn't sound like a huge technical challenge unless you're operating at scale. So my first question would be why do you have a large codebase so with so few customers?

[reply](#)

duskdozer 2 days ago | root | parent | next [-]

He's 5-7xing code output with the help of ~100% AI. More lines. More vibes. More velocity. Rocketship emoji.

[reply](#)

recroad 2 days ago | root | parent | next [-]

Less vibes. In SDD you have to meticulously review your specs.

[reply](#)

_se 1 day ago | root | parent | next [-]

Really, REALLY make no mistakes!!!

[reply](#)

galexymending 2 days ago | prev | next [-]

I gave it a shot, but won't be using it going forward. It requires a waterfall process. And, I found it difficult, and in some cases impossible, to adjust phases/plans when bugs or changes in features arise. The execution prompts didn't do a good job of steering the code to be verified while coding and relies on the user to manually test at the end of each phase.

[reply](#)

obsidianbases1 2 days ago | prev | next [-]

> If you know clearly what you want

This is the real challenge. The people I know that jump around to new tools have a tough time explaining what they want, and thus how new tool is better than last tool.

[reply](#)

boringg 2 days ago | parent | next [-]

What do you think drives the tooling ecosystem aside from VC dollars?

[reply](#)

jauntywundrkind 2 days ago | root | parent | next [-]

These are incredible new superpowers. The LLMs let us do far far more than we could before. But it creates information glut, doesn't come with in built guards to prevent devolution from setting in. It feels unsurprising but also notable that a third of what folks are suddenly building is harness/prompting/coordination systems, because it's all trying to adapt & figure out process shapes for using these new superpowers well in.

There's some VC money interest but I'd classify more than 9 / 10ths of it as good old fashioned wildcat open source interest. Because it's fascinating and amazing, because it helps us direct our attention & steer our works.

And also it's so much more approachable and interesting, now that it's all tmux terminal stuff. It's so much more direct & hackable than, say, wading into vscode extension building, deep in someone else's brambly thicket of APIs, and where the skeleton is already in place anyhow, where you are only grafting little panes onto the experience rather than recasting the experience. The devs suddenly don't need or care for or want that monolithic big UI, and have new soaring freedom to explore something much nearer to them, much more direct, and much more malleable: the terminal.

There's so many different forms of this happening all at once. Totally different topic, but still in the same broad area, submitted just now too: Horizon, an infinite canvas for trrminals/AI work.

<https://github.com/peters/horizon> <https://news.ycombinator.com/item?id=47416227>

[reply](#)

paddy_m 2 days ago | prev | next [-]

Has anything like this been built?

I want a system that enforces planning, tests, and adversarial review (preferably by a different company's model). This is more for features, less for overall planning, but a similar workflow could be built for planning.

1. Prompt 2. Research 3. Plan (including the tests that will be written to verify the feature) 4. adversarial review of plan 5. implementation of tests, CI must fail on the tests 6. adversarial review verifying that the tests match with the plan 7. implementation to make the tests pass. 8. adversarial PR review of implementation

I want to be able to check on the status of PRs based on how far along they are, read the plans, suggest changes, read the tests, suggest changes. I want a web UI for that, I don't want to be doing all of this in multiple terminal windows.

A key feature that I want is that if a step fails, especially because of adversarial review, the whole PR branch is force pushed back to the previous state. so say #6 fails, #5 is re-invoked with the review information. Or if I come to the system and a PR is at #8, and I don't like the plan, then I make some edits to the plan (#3), the PR is reset to the git commit after the original plan, and the LLM is reinvoked with either my new plan or more likely my edits to the plan, then everything flows through again.

I want to be able to sit down, tend to a bunch of issues, then come back in a couple of hours and see progress.

I have a design for this of course. I haven't implemented it yet.

[reply](#)

dboreham 2 days ago | parent | next [-]

Similar ideas have been kicked around over here. One problem is that this seems like a set of features for GitHub rather than a stand-alone product (so no way to make money from it).

[reply](#)

paddy_m 2 days ago | root | parent | next [-]

I'm not concerned about making money from it, I just want to use it. I'd like to check to see if I'm re-inventing the wheel. I'm curious if others would like a similar experience.

[reply](#)

giancarlostorio 2 days ago | prev | next [-]

Nice, I like the UI more than mine, I built a similar tool out of minor frustrations with some design choices in Beads, mine uses SQLite exclusively instead of git or hard files, been using it for all my personal projects, but haven't gone back to try and refine what I have a little more. One thing a lot of these don't do that I added to mine is synching to and from GitHub. I want people to see exactly what my local tasks are, and if they need to pull one down to work on.

I think the secret sauce is talk to the model about what you want first, make the plan, then when you feel good about the spec, regardless of tooling (you can even just use a simple markdown file!) you have it work on it. Since it always has a file to go back to, it can never 'forget' it just needs to remember to review the file. The more detail in the file, the more powerful the output.

Tell your coding model: how you want it, what you want, and why you want it. It also helps to ask it to poke holes and raise concerns (bypass the overly agreeable nature of it so you don't waste time on things that are too complex).

I love using Claude to prototype ideas that have been in my brain for years, and they wind up coming out better than I ever envisioned.

[reply](#)

joshribakoff 2 days ago | parent | next [-]

The "secret sauce"? RPI (research plan implement) is far from a secret concept. Heck, it's in the official docs! Not secret.

"never forgetting"? It's still a probabilistic model.

You're framing it like you discovered some secret techniques that eludes 100s of millions of users and overcame the architectural limits of the model.

[reply](#)

anentropic 2 days ago | prev | next [-]

I have been using this a lot lately and ... it's good.

Sometimes annoying - you can't really fire and forget (I tend to regret skipping discussion on any complex tasks). It asks a lot of questions. But I think that's partly why the results are pretty good.

The new `/gsd:list-phase-assumptions` command added recently has been a big help there to avoid needing a Q&A discussion on every phase - you can review and clear up any misapprehensions in one go and then tell it to plan -> execute without intervention.

It burns quite a lot of tokens reading and re-reading its own planning files at various times, but it manages context effectively.

Been using the Claude version mostly. Tried it in OpenCode too but is a bit buggy.

They are working on a standalone version built on pi.dev <https://github.com/gsd-build/gsd-2> ...the rationale is good I guess, but it's unfortunate that you can't then use your Claude Max credits with it as has to use API.

[reply](#)

seneca 2 days ago | prev | next [-]

I've tried several of these sorts of things, and I keep coming away with the feeling that they are a lot of ceremony and complication for not much value. I appreciate that people are experimenting with how to work with AI and get actual value,

but I think pretty much all of these approaches are adding complexity without much, or often any, gain.

That's not a reason to stop trying. This is the iterative process of figuring out what works.

[reply](#)

dfltr 2 days ago | prev | next [-]

GSD has a reputation for being a token burner compared to something like Superpowers. Has that changed lately? Always open to revisiting things as they improve.

[reply](#)

melvinroest 2 days ago | prev | next [-]

If you want some context about spec-driven development and how it could be used with LLMs I recommend [1]. Having some background like helps me to understand tools like this a bit more.

[1] <https://www.riaanzoetmulder.com/articles/ai-assisted-program...>

[reply](#)

visarga 2 days ago | prev | next [-]

I did a similar system myself, then I run evals on it and found that the planning ceremony is mostly useless, claude can deal with simple prose, item lists, checkbox todos, anything works. The agent won't be a better coder for how you deliver your intent.

But what makes a difference is running plan review and work review agent, they fix issues before and after work. Both pull their weight but the most surprising is the plan-review one. The work review judge reliably finds bugs to fix, but not as surprising in its insights. But they should run from separate subagents not main one because they need a fresh perspective.

Other things that matter are 1. testing enforcement, 2. cross task project memory. My implementation for memory is a combination of capturing user messages with a hook, append only log, and keeping a compressed memory state of the project, which gets read before work and updated after each task.

[reply](#)

arjie 2 days ago | prev | next [-]

I could not produce useful output from this. It was useful as a rubber duck because it asks good motivating questions during the plan phase, but the actual implementation was lacklustre and not worth the effort. In the end, I just have Claude Opus create plans, and then I have it write them to memory and update it as it goes along and the output is better.

[reply](#)

hatmanstack 2 days ago | parent | next [-]

No brother, the Claude plans aren't the right path, they're for hobbyists.

[reply](#)

arjie 2 days ago | root | parent | next [-]

Okay, I'll give these another shot. Perhaps I just haven't figured out how to use them right.

[reply](#)

hatmanstack 2 days ago | root | parent | next [-]

I don't know brother, I don't use them, they may be great they may suck. What I've found is that adding peripherals always creates more problems. If you aren't using Claude for professional work then just sticking with the factory plan mode probably works. If not, look into creating your own Claude skills, try to understand how prompt pipelines work and it will unlock a ton of automation for you. Not just for coding.

[reply](#)

arjie 2 days ago | root | parent | next [-]

I do use Claude for professional work, but I suspect I don't know enough to be able to get anything useful out of your advice since I don't know how to add my own Claude skills that form a prompt pipeline without adding peripherals. I'll just have to catch this part of the whole thing as a late adopter, I suppose. Ah well, thanks for the help.

[reply](#)

hatmanstack 2 days ago | root | parent | next [-]

no worries, if you want a simple idea of how a prompt pipeline can work so you can create your own...<https://github.com/hatmanstack/claude-forge>

[reply](#)

jankhg 2 days ago | prev | next [-]

Apart from GSD and superpowers, there's another system, called PAUL [1]. It apparently requires fewer tokens compared to GSD, as it does not use subagents, but keeps all in one session. A detailed comparison with GSD is part of the repo [2].

[1] <https://github.com/ChristopherKahler/paul>

[2] <https://github.com/ChristopherKahler/paul/blob/main/PAUL-VS-...>

[reply](#)

prakashrj 2 days ago | parent | next [-]

Would love to migrate from GSD and try, if there is community around it.

[reply](#)

theodorewiles 2 days ago | prev | next [-]

I think the research / plan / execute idea is good but feels like you would be outsourcing your thinking. Gotta review the plan and spend your own thinking tokens!

[reply](#)

jdwyah 1 day ago | prev | next [-]

I used GSD for a bit. It was helpful for a side project where I constantly forgot where I was in implementation. Helpful to be able to just say "Do the next thing"

I would imagine that for a non-engineer trying to code it would be quite useful / deliver a better result / less liable to end up in total mess. But for experienced engineers it quickly felt like overkill / claude itself just gets better and better. Particularly once we got agent swarms I left GSD and don't think I'll be back. But I would recommend it to non coders trying to code.

[reply](#)

j45 1 day ago | parent | next [-]

In your experience do you think something like this could feed into agent swarms pretty well?

[reply](#)

smusamashah 2 days ago | prev | next [-]

There should be an "Examples" section in projects like this one to show what has actually been made using it. I scrolled to the end and was really expecting an example the way it's being advertised.

If it was game engine or new web framework for example there would be demos or example projects linked somewhere.

[reply](#)

randomthought12 2 days ago | prev | next [-]

I tried this but it creates a lot of content inside the repository and I don't like that. I understand these tools need to organize their context somewhere to be efficient but I feel that it just pollutes my space.

If multiple people work with different AI tools on the same project, they will all add their own stuff in the project and it will become messy real quick.

I'll keep superpowers, claude-mem, context7 for the moment. This combination produces good results for me.

[reply](#)

rdtsc 2 days ago | prev | next [-]

> GSD is designed for frictionless automation. Run Claude Code with: `claude --dangerously-skip-permissions`

Is this supposed to run in a VM?

[reply](#)

mobrienv 1 day ago | prev | next [-]

All you need is a good spec and a hat-based ralph-orchestrator loop

<https://github.com/mikeyobrien/ralph-orchestrator>

[reply](#)

BTAQA 2 days ago | prev | next [-]

Built my first SaaS as a frontend dev with no backend experience using a similar approach. The key shift was treating Claude Code as a senior developer who needs clear specs, not a magic box. The more precise the context and requirements, the better the output. Vague prompts produce vague code.

[reply](#)

jcmontx 2 days ago | prev | next [-]

My experience with this library has been underwhelming sadly. I have a better experience going raw with any cli agent

[reply](#)

lemax 2 days ago | prev | next [-]

I'm still stuck on superpowers. Can't seem to get better plans out of native claude planning - superpowers ensures I have a reviewed design that actually matches my mental model. Typical claude planning doesn't confirm assumptions sufficiently for

my weak brain dumps/poorly spec'd tickets.

[reply](#)

soorya3 2 days ago | prev | next [-]

The only tool you need is the one that saves tokens... the one that saves tokens ... the one that saves tokens. Currently I don't know any.

Claude code itself consumes lot of tokens when not needed. I have to steer it a lot while building large applications.

[reply](#)

redmattred 2 days ago | parent | next [-]

Here's one: <https://www.rtk-ai.app/> (<https://github.com/rtk-ai/rtk>)

[reply](#)

ricardo_lien 2 days ago | prev | next [-]

I tried it after watching the video demo from the repo creator, and it looked quite impressive at first. And I decided to rebuild my side project with this, but after a few days I realized that it was not for me. It's way too much of a black box for me as an engineer, not a prompter.

[reply](#)

chriss395 2 days ago | prev | next [-]

I'm curious if anyone has used this (or similar) to build a production system?

I'm facing increasing pressure from senior executives who think we can avoid the \$\$\$ B2B SaaS by using AI to vibe code a custom solution. I love the idea of experimenting with this but am horrified by the first-ever-case being a production system that is critical to the annual strategic plan. :-/

[reply](#)

prakashrj 2 days ago | parent | next [-]

I would love to take up that challenge. With what I have learnt so far, I am raring to get opportunities to make custom solutions.

[reply](#)

justacatbot 2 days ago | prev | next [-]

The spec-first approach is underrated. Treating the spec as a living artifact the AI can reference across sessions is something I've been experimenting with too. The main challenge is keeping specs short enough to actually stay current.

[reply](#)

yoavsha1 2 days ago | prev | next [-]

How come we have all these benchmarks for models, but none whatsoever for harnesses / whatever you'd call this? While I understand assigning "scores" is more nuanced, I'd love to see some website that has a catalog of prompts and outputs as produced with a different configuration of model+harness in a single attempt

[reply](#)

Andrei_dev 2 days ago | prev | next [-]

250K lines in a month — okay, but what does review actually look like at that volume?

I've been poking at security issues in AI-generated repos and it's the same thing: more generation means less review. Not just logic — checking what's in your .env, whether API routes have auth middleware, whether debug endpoints made it to prod.

You can move that fast. But "review" means something different now. Humans make human mistakes. AI writes clean-looking code that ships with hardcoded credentials because some template had them and nobody caught it.

All these frameworks are racing to generate faster. Nobody's solving the verification side at that speed.

[reply](#)

kace91 2 days ago | parent | next [-]

Code is a cost. It seems everyone's forgotten.

Saying "I generated 250k lines" is like saying "I used 2500 gallons of gas". Cool, nice expense, but where did you get? Because it it's three miles, you're just burning money.

250k lines is roughly SQLite or Redis in project size. Do you have SQLite-maintaining money? Did you get as far as Redis did in outcomes?

[reply](#)

prakashrj 2 days ago | root | parent | next [-]

Openclaw was mostly build by AI. It had 400K lines of code.

[reply](#)

sarchertech 2 days ago | root | parent | next [-]

You didn't answer what does you 250k lines do? How much money does it make? How many users does it have?

[reply](#)

knes 2 days ago | root | parent | next [-]

what if you move from reviewing the code to reviewing the spec?

[reply](#)

sarchertech 2 days ago | root | parent | next [-]

That's like asking why don't we switch from reviewing PRs to reviewing jira tickets.

There's probably a world where you could do that if the spec was written in a formal language with no ambiguity and there was a rigorous system for translating from spec to code sure.

[reply](#)

duskdozer 2 days ago | root | parent | next [-]

Hm, that's an interesting concept. What if we were able to create an unambiguous, rigorous specification language for creating prompts so that we could get consistent and predictable output from AI? Maybe we could call it a "prompt programming language" or something

[reply](#)

ccanassa 1 day ago | root | parent | next [-]

That exists, it's called code.

[reply](#)

CuriouslyC 2 days ago | parent | prev | next [-]

I've been trying to beat this drum for a minute now. Your code quality is a function of validation time, and you have a finite amount of that which isn't increased by better orchestration.

My rant about this: <https://sibylline.dev/articles/2026-01-27-stop-orchestrating...>

[reply](#)

ManWith2Plans 2 days ago | parent | prev | next [-]

I agree with this to some degree. Agents often stub and take shortcuts during implementation. I've been working on this problem a little bit with open-artisan which I published yesterday (<https://github.com/yehudacohen/open-artisan>).

Rather than having agents decide to manage their own code lifecycle, define a state machine where code moves from agent to agent and isolated agents critique each others code until the code produced is excellent quality.

This is still a bit of a token hungry solution, but it seems to be working reasonably well so far and I'm actively refining it as I build.

Not going to give you formal verification, but might be worth looking into strategies like this.

[reply](#)

jtbetz22 2 days ago | parent | prev | next [-]

I have been ~obsessed~ with exactly this problem lately.

We built AI code generation tools, and suddenly the bottleneck became code review. People built AI code reviewers, but none of the ones I've tried are all that useful - usually, by the time the code hits a PR, the issues are so large that an AI reviewer is too late.

I think the solution is to push review closer to the point of code generation, catch any issues early, and course-correct appropriately, rather than waiting until an entire change has been vibe-coded.

[reply](#)

prakashrj 2 days ago | parent | prev | next [-]

You can AI to audit and review. You can put constraints that credentials should never hit disk. In my case, AI uses sed to read my env files, so the credentials don't even show up in the chat.

Things have changed quite a bit. I hope you give GSD a try yourself.

[reply](#)

lielcohen 2 days ago | parent | prev | next [4 more]

LoganDark 2 days ago | prev | next [-]

This seems like something I'd want to try but I am wholly opposed to `npx` being the sole installation mechanism. Let me install it as a plugin in Claude Code. I don't want `npx` to stomp all over my home directory / system configuration for this, or auto-find directories or anything like that.

[reply](#)

scuff3d 2 days ago | [prev](#) | [next](#) [-]

Oh boy, if anyone thought productivity hacks, ultra optimized workflows, and "personal knowledge management" systems could get ridiculous, they haven't seen anything yet. This is gonna be the new thing people waste time on now instead of their NeoVim config.

[reply](#)

MeetingsBrowser 2 days ago | [prev](#) | [next](#) [-]

I've tried it, and I'm not convinced I got measurably better results than just prompting claude code directly.

It absolutely tore through tokens though. I don't normally hit my session limits, but hit the 5-hour limits in ~30 minutes and my weekly limits by Tuesday with GSD.

[reply](#)

testycool 2 days ago | [parent](#) | [next](#) [-]

Same experience on multiple occasions.

[reply](#)

bubblerrme 2 days ago | [prev](#) | [next](#) [-]

The spec-driven approach resonates. I've found that the quality of the initial context you feed to AI coding tools determines everything downstream. Vague specs produce vague code that needs constant correction.

One pattern that's worked well for me: instead of writing specs manually, I extract structured architecture docs from existing systems (database schemas, API endpoints, workflow logic) and use those as the spec. The AI gets concrete field names, actual data relationships, and real business logic — not abstractions. The output quality jumps significantly compared to hand-written descriptions.

The tricky part is getting that structured context in the first place. For greenfield projects it's straightforward. For migrations or rewrites of existing systems, it's the bottleneck that determines whether AI-assisted development actually saves time or just shifts the effort from coding to prompt engineering.

[reply](#)

duskdozer 2 days ago | [parent](#) | [next](#) [-]

>Don't post generated comments or AI-edited comments. HN is for conversation between humans.

<https://news.ycombinator.com/newsguidelines.html>

[reply](#)

hermanzegerman 2 days ago | [prev](#) | [next](#) [-]

For me it was awesome. I needed a custom Pipeline for Preprocessing some Lab Data, including Visualization and Manipulation and it got me exactly what I wanted, as opposed to Codex Plan Mode, which just burned my weekly quota and produced Garbage

[reply](#)

hexnuts 2 days ago | [prev](#) | [next](#) [-]

You are missing one important bit. Semantic Gravity Sieves. Important data in the metadata collapses together, allowing grouped indexing. Something like a DAG allows the logic to be addressed consistently.

[reply](#)

jessepcc 2 days ago | [prev](#) | [next](#) [-]

With the coding slot machine, I prefer move fast and start over if anything goes off track. Maybe the amount of token spent with several iterations is similar to using a more well planned system like GSD.

[reply](#)

spaceman_2020 2 days ago | [prev](#) | [next](#) [-]

I've been using GSD for all my dev projects

Honestly a fantastic harness right out of the box. Give it a good spec and it can easily walk you through fairly complex apps

[reply](#)

dhorthy 2 days ago | [prev](#) | [next](#) [-]

it is very hard for me to take seriously any system that is not proven for shipping production code in complex codebases that have been around for a while.

I've been down the "don't read the code" path and I can say it leads nowhere good.

I am perhaps talking my own book here, but I'd like to see more tools that brag about "shipped N real features to production" or "solved Y problem in large-10-year-old-codebase"

I'm not saying that coding agents can't do these things and such tools don't exist, I'm just afraid that counting 100k+ LOC that the author didn't read kind of fuels the "this is all hype-slop" argument rather than helping people discover the ways that coding agents can solve real and valuable problems.

[reply](#)

knes 2 days ago | parent | next [-]

Agreed.this paper studied 33k+ agent-authored PRs on GitHub (<https://arxiv.org/pdf/2601.15195>)

#1 rejection reason: missing context. 80% needed human fixes. Agents can write code fine. They just don't know what "done" looks like in your codebase.

Count successful merges into repos with real history instead of LOC and the hard part is specification, not execution.

Wrote about this topic @ <https://www.augmentcode.com/blog/the-end-of-linear-work>

[reply](#)

davispeck 2 days ago | prev | next [-]

This looks like moving context from prompts into files and workflows.

Makes sense for consistency, but also shifts the problem:

how do you keep those artifacts in sync with the actual codebase over time?

[reply](#)

OpenDQV 2 days ago | prev | next [-]

I haven't read everything in here but think this will be very useful going forwards. love the name btw! GSD!

[reply](#)

DIVx0 2 days ago | prev | next [-]

I've tried GSD several times. I actually like the verbosity and it's a simple chore for Claude to refresh project docs from GSD planning docs.

Like most spec driven development tools, GSD works well for greenfield or first few rounds of "compound engineering." However, like all others, the project gets too big and GSD can't manage to deliver working code reliably.

Agents working GSD plans will start leaving orphans all over, it won't wire them up properly because verification stages use simple lexical tools to search code for implementation facts. I tried giving GSD some ast aware tools but good luck getting Claude to reliably use them.

Ultimately I put GSD back on the shelf and developed my own "property graph" based planner that is closer to Claude "plan mode" but the design SOT is structured properties and not markdown. My system will generate docs from the graph as user docs. Agents only get tasked as my "graph" closes nodes and re-sorts around invariants, then agents are tasked directly.

[reply](#)

lifeisstillgood 2 days ago | parent | next [-]

Can you expand on that at all (or point to some reading on how Claude plan mode works etc?)

I think I have to get my head around a lot more than I think

[reply](#)

DIVx0 2 days ago | root | parent | next [-]

claude code (CC) plan mode: In a normal CC window hit "shift-tab" until you see "plan mode on" in the lower left hand corner of the TUI.

Now all you really have to do is chat with claude about what you're thinking about building.

In plan mode, claude can't edit anything and has some extra "you're an expert at planning!" prompts prepended to your initial message in plan mode.

And then either when you're ready or Claude thinks the "plan" is gelling, it'll suggest stopping and let it write up a detailed plan. CC will dispatch some "planning agents" with prompts that your 'main' CC has crafted for that agent to plan for within the context of your conversation and what parts of the codebase it should look to integrate/explore.

Once all that is done, it will display it to you and then offer to "clear context and implement" - where it will just get to work. Or it will offer to go back to chatting and resolve whatever was misunderstood, or if you had a new idea you wanted to mix in.

These plans are saved as markdown in your .claude/plans directory.

Plan mode is handy on the on-off. But if you enter another plan mode, thinking claude would learn from, or build off a previous plan spec, it won't unless you explicitly say something like "read previous plan <path to plan file> and re-use the scaffolding directives for this new project"

[reply](#)

loveparade 2 days ago | prev | next [-]

"I am a super productive person that just wants to get shit done"

Looked at profile, hasn't done or published anything interesting other than promoting products to "get stuff done"

This is like the TODO list book gurus writing about productivity

[reply](#)

dasil003 2 days ago | parent | next [-]

Looking for 5 seconds at the github profile I see a bunch of music-related stuff, and also a bunch of contributions to private repos that we have no idea what they are. I get the productivity guru anti-pattern, but I honestly don't know what you're looking at that merits this kind of reflexive personal attack.

[reply](#)

thr0waway001 2 days ago | prev | next [-]

At the risk of sounding stupid what does the author mean by: "I'm not a 50-person software company. I don't want to play enterprise theatre." ?

[reply](#)

jdthedisciple 2 days ago | parent | next [-]

Seems fairly obvious: Some agent harnesses play enterprise theater by creating jira-type tickets for you and moving them around silly swim lanes, instead of, of course, just simply getting sh!t done.

[reply](#)

thr0waway001 2 days ago | root | parent | next [-]

Wasn't obvious to me so I asked.

But I guess if I go by what you're saying I suppose it makes sense for it not to do a bunch of things you didn't ask it to do.

[reply](#)

saaaaaam 2 days ago | parent | prev | next [-]

No idea but doesn't it sound GREAT and filled with portentous meaning? Don't be an enterprise clown! Be a gutsy hustle guy like me! Down with enterprise theatre, long live the vibe jam!

[reply](#)

bobtheborg 2 days ago | parent | prev | next [-]

The author of that page seems to mostly be AI, not a human.

[reply](#)

canadianm 2 days ago | prev | next [-]

I use Oh-My-OpenCode (Now called Oh-My-OpenAgent), but it's effectively the same as GSD, but better imo

[reply](#)

prakashrj 2 days ago | prev | next [-]

With GSD, I was able to write 250K lines of code in less than a month, without prior knowledge of claude.

[reply](#)

rsoto2 2 days ago | parent | next [-]

I could copy 250k lines from github.

Faster than using ai. Cheaper. Code is better tested/more secure. I can learn/build with other humans.

[reply](#)

prakashrj 2 days ago | root | parent | next [-]

This is how I test my code currently.

1. Backend unit tests – fast in-memory tests that run the full suite in ~5 seconds on every save.
2. Full end-to-end tests – automated UI tests that spin up a real cloud server, run through the entire user journey (provision → connect → manage → teardown), and verify the app behaves correctly on all supported platforms (phone, tablet, desktop).
3. Screenshot regression tests – every E2E run captures named screenshots and diffs them against saved baselines. Any unintended UI change gets caught automatically.

[reply](#)

indigodaddy 2 days ago | root | parent | next [-]

Check out exe.dev/Shelley web agent it facilitates much of what you describe by default.

[reply](#)

dominotw 2 days ago | root | parent | next [-]

yea i am not going to checkout your shitty vibecoded project.

Can we pls stop this.

[reply](#)

indigodaddy 2 days ago | root | parent | next [-]

Lol, not my project, and you shouldn't make assumptions, you have no clue what you are talking about

[reply](#)

rsoto2 2 days ago | root | parent | prev | next [-]

sounds like your only measure of good tests is how quickly the llm can produce and run them. not a good metric.

LOL screenshot regression. You're still not a dev buddy read some books

[reply](#)

prakashrj 2 days ago | root | parent | prev | next [-]

I was not a app developer before, but a systems engineer with devops experience. But I learnt a lot about apple development, app store connect and essential became a app developer in a month. I don't think I can learn so quickly with other humans help.

[reply](#)

icedchai 2 days ago | root | parent | next [-]

You might be surprised. In 2008, when the App Store first came out, I became an iPhone app developer after reading one book. I already knew C, so Objective C wasn't a big leap.

Between my own apps and consulting work, I had a pretty good side business. Like everything else though, those days didn't last forever. But there was a lot of easy money early on.

[reply](#)

0x696C6961 2 days ago | root | parent | prev | next [-]

If you lost access to AI would you be able to continue development on your app?

[reply](#)

prakashrj 2 days ago | root | parent | next [-]

Goal is to build something that will have value. Once it has value, I can hire a team or open source it, if AI ceases to exist in this world.

[reply](#)

tkiolp4 2 days ago | parent | prev | next [-]

That sounds awful.

I got a promotion once for deleting 250K lines of code in less than a month. Now that sounds better

[reply](#)

prakashrj 2 days ago | root | parent | next [-]

I get it now. Hopefully the utility of it will eventually bring some value. Maybe Utility and corresponding LOC should help you assess my work. Since I didn't share what I have, I can see people getting alarmed at 250K lines of code.

[reply](#)

wslh 2 days ago | parent | prev | next [-]

250K? Could you expand your experience with details about your project and the lessons and issues you found?

[reply](#)

prakashrj 2 days ago | root | parent | next [-]

A self-hosted VPN server manager: a TypeScript/Hono backend that runs on your own VPS, paired with a SwiftUI iOS/macOS app. It lets you provision cloud servers across multiple providers (Hetzner, DigitalOcean, Vultr), manage them via a Tailscale-secured connection with TLS pinning, and control an OpenClaw gateway.

I will open source it soon in few weeks, as I have still complete few more features.

[reply](#)

icedchai 2 days ago | root | parent | next [-]

This does not feel like 250K lines of complexity. Have you looked at any of the code at all? You likely have mass duplication, copy-pasta everywhere.

[reply](#)

prakashrj 2 days ago | root | parent | next [-]

I didn't look at code. In addition to code, I have CI and CD built in. I becomes hard add features after a while, if you cannot have built in CI/CD that will catch regression.

[reply](#)

icedchai 2 days ago | root | parent | next [-]

You didn't look at the code, so you don't know what you're really working with. Maybe it's total slop. This is concerning since you're dealing with security and presumably API keys to third-party platforms.

[reply](#)

tkiopl4 2 days ago | root | parent | prev | next [-]

Please don't.

[reply](#)

prakashrj 2 days ago | root | parent | next [-]

It's good advice. I will only open source, if it has utility.

[reply](#)

toastal 2 days ago | root | parent | prev | next [-]

Please do. Poison the training.

[reply](#)

prakashrj 2 days ago | root | parent | prev | next [-]

It's important to build a local dev environment that GSD can iterate on. Once I have done that, I just discuss with GSD and few hours later features land.

[reply](#)

dominotw 2 days ago | parent | prev | next [-]

yes vibecoding is fun.

[reply](#)

desireco42 2 days ago | prev | next [-]

I honestly tried this a while back, unless this is something else, this was completely not very much useful thing.

If I remember correctly, it created a lot of changes, spent a lot of time doing something and in the end this was all smoke and mirrors. If I would ever use something like this, I would maybe use BMad, which suffers from same issues, like Speckit and others.

I don't know if they have some sponsorship with bunch of youtubers who are raving how awesome this is... without any supporting evidence.

Anyhow, this is my experience. Superpowers on the other hand were quite useful so far, but I didn't use them enough to have to claim anything.

[reply](#)

ibrahim_h 2 days ago | prev | next [-]

The README recommends --dangerously-skip-permissions as the intended workflow. Looking at gsd-executor.md you can see why — subagents run node gsd-tools.cjs, git checkout -b, eslint, test runners, all generated dynamically by the planner. Approving each one kills autonomous mode.

There is a gsd-plan-checker that runs before execution, but it only verifies logical completeness — requirement coverage, dependency graphs, context budget. It never looks at what commands will actually run. So if the planner generates something destructive, the plan-checker won't catch it because that's not what it checks for. The gsd-verifier runs after execution, checking whether the goal was achieved, not whether anything bad happened along the way. In /gsd:autonomous this chains across all remaining phases unattended.

The granular permissions fallback in the README only covers safe reads and git ops — but the executor needs way more than that to actually function. Feels like there should be a permission profile scoped to what GSD actually needs without going full skip.

[reply](#)

Relisora 2 days ago | prev | next [-]

Did anyone compare it with everything-claude-code (ECC)?

[reply](#)

jatora 2 days ago | prev | next [-]

Another heavily overengineered AND underengineered abomination. I'm convinced anyone who advocates for these types of tools would find just as much success just prompting claude code normally and taking a little bit to plan first. Such a waste of time to bother with these tools that solve a problem that never existed in the first place.

[reply](#)

gverrilla 2 days ago | prev | next [-]

Unbelievably slow, not worth it at all.

[reply](#)

notepad0x90 2 days ago | prev | next [-]

if you want to charge for this, or even if you don't and you want people in old & boring companies to use it, imagine a developer or engineer having this conversation with management/bureaucrats:

"I want to use 'get shit done' as part of my project"

These days, it's not a big deal at all at most places. But there are places where it will raise an eye brow. I'm not saying change it's name, and you've probably considered this already, but I would like to suggest the meaning of GSD tongue-in-cheek perhaps? Whatever, a kick-ass project either way.

[reply](#)

liampulles 2 days ago | prev | next [-]

Please stop using the term prompt engineering, context engineering, etc. to define formatting the text that we send an LLM.

Its already quite debatable whether software developers should be called software engineers, but this is just ridiculous.

[reply](#)

barbazoo 2 days ago | parent | next [-]

Why?

[reply](#)

yrds96 1 day ago | prev | next [-]

Let me guess. Yeat another bunch of md files trying to fix bad prompts from unskilled programmers?

[reply](#)

LetsGetTechnicl 2 days ago | prev | next [-]

Get slop done

[reply](#)

noduerme 2 days ago | prev | next [-]

Question for people who have spent more time than I have wrangling agents to manage other agents:

I've been using a Claude Pro plan just as a code analyzer / autocomplete for a year or so. But I recently decided to try to rewrite a very large older code base I own, and set up an AI management system for it.

I started this last week, after reading about paperclip.ing. But my strategy was to layer the system in a way I felt comfortable with. So I set up something that now feels a bit like a rube goldberg machine. What I did was, set up a clean box and give my Claude Pro plan root access to it. Then set up openclaw on that box, but not with root... so just in case it ran wild, I could intervene. Then have openclaw set up paperclip.ing.

The openclaw is on a separate Claude API account and is already costing what seems like way too many tokens, but it *does* have a lot of memory now of the project, and in fairness, for the \$150 I've spent, it has rewritten an enormous chunk of the code in a satisfactory way (with a lot of oversight). I do like being able to whatsapp with it - that's a huge bonus.

But I feel like maybe this a pretty wasteful way of doing things. I've heard maybe I could just run openclaw through my Claude Pro plan, without paying for API usage. But I've heard that Anthropic might be shutting down that OAuth pathway. I've also heard people saying openclaw just thoroughly sucks, although I've been pretty impressed with its results.

The general strategy I'm taking on this is to have Claude read the old codebase side by side with me in VSCode, then prepare documents for openclaw to act on as editor, then re-evaluate; then have openclaw produce documents for agent roles in Paperclip and evaluate them.

Am I just wasting my money on all these API calls? \$150 so far doesn't seem bad for the amount of refactoring I've gotten, across a database and back and front end at the same time, which I'm pretty sure Claude Pro would not have been able to handle without much more file-by-file supervision. I'm slightly afraid now to abandon the memory I've built up with openclaw and switch to a different tool. But hey, maybe I should just be doing this all on the Claude Pro CLI at this point...?

Looking for some advice before I try to switch this project to a different paradigm. But I'm still testing this as a structure, and trying to figure out the costs.

[Edit: I see so many people talking about these lighter-weight frameworks meant for driving an agent through a large, long-running code building task... like superpowers, GSD, etc... which to me as a solo coder sound very appealing if I were building a new project. But for taking 500k LOC and a complicated database and refactoring the whole thing into a headless version that can be run by agents, which is what I'm doing now, I'm not sure those are the right tools; but at the same time, *I never heard anyone say openclaw was a great coding assistant* -- all I hear about it being used for is, like, spamming Twitter or reading your email or ordering lunch for you. But I've *only* used it as a code-manager, not for any daily tasks, and I'm pretty impressed with its usefulness at that...]

[reply](#)

wyre 2 days ago | parent | next [-]

Ya, openclaw is overkill for rewriting a codebase, especially when you're paying API costs.

I developed my own task tracker (github.com/kfcafe/beans), i'm not sure how portable it is; it's been a while since i've used it in claude code. I've been using pi-coding-agent the past few months, highly recommend, it's what's openclaw is built on top of. Anthropic hasn't shut down Oauth, they just say that it's banned outside of Claude Code. I'd recommend installing pi, tell it what you were doing with openclaw and have it port all of the information over to the installation of pi.

you could also check out ralph wiggum loops, could be a good way to rewrite the codebase. just write a prompt describing what you want done, and write a bash loop calling claude's cli pointed at the prompt file. the agent should run on a loop until until you decide to stop it. also not the most efficient usage of tokens, but at least you will be using Claude Pro and not spending money on API calls.

[reply](#)

noduerme 2 days ago | root | parent | next [-]

I'm kinda doing this in a back-and-forth way over each section with openclaw, and one nice thing is that I've got it including the chat log for changes with each commit. I'm happy about how it's handled my personality as needing to understand all the changes it's making before committing. So I kind of want something interactive like that -- this isn't a codebase I can trust an LLM to just fire and forget (as evidenced by some massive misunderstandings about rewiring message strings and parameter names like "_meta" and ".meta" and "_META" that meant completely different things which the LLM accidentally crossed and merged at some point, before I caught it and forced it to untangle the whole mess -- which it only did well because there were good logs).

I sort of *do* need something with persistent memory and personality... or a way to persist it without spending a lot of time trying to bring it back up to speed... it's not exactly specific tasks being tracked, I need it to have a fairly good grasp on the entire ecosystem.

[reply](#)

wyre 2 days ago | root | parent | next [-]

how big is the codebase? how often is the agent writing to memory? you might be able to get away with just appending it to the project's CLAUDE.md? you might also want to check out <https://github.com/probelabs/probe>

[reply](#)

noduerme 2 days ago | root | parent | next [-]

Hm. That looks a lot more granular, which is interesting... I'm not sure it would help me on this.

The codebase is small enough that I can basically go and find all the changes the LLM executed with each request, and read them with a very skeptical eye to verify that they look sane, and ask it why it did something or whether it made a mistake if anything smells wrong. That said, the code I'm rewriting is a genetic algorithm / evaluation engine I wrote years ago, which itself writes code that it then evaluates; so the challenge is having the LLM make changes to the control structure, with the aim of having an agent be able to run the system at high speed and read the result stream through a headless API, without breaking either the writing or evaluation of the code that the codebase itself is writing and running. Openclaw has a surprisingly good handle on this now, after a very very very long running session, but most of the problems I'm hitting still have to do with it not understanding that modifying certain parameters or names could cause downstream effects in the output (eval code) or input (load files) of the system as it's evolving.

[reply](#)

greenchair 2 days ago | prev | next [2 more]

tki0lp4 2 days ago | prev [-]

The whole gsd/agents folder is hilarious. Like a bunch of MD that never breaks. How do you is it minimally correct? Subjective prose. Sad to see this on the frontpage

[reply](#)

[Guidelines](#) | [FAQ](#) | [Lists](#) | [API](#) | [Security](#) | [Legal](#) | [Apply to YC](#) | [Contact](#)

Search: